ChatGPT

Summary of Prior Model Survey

The previous internal survey compared five model families for barline detection and measure structure in OMR: the existing **HOMR/Oemer pipeline**, a **YOLO-based object detector**, a **Mask R-CNN instance segmenter**, a **DETR transformer with notation graph**, and an **end-to-end seq2seq OMR model**. Key findings included:

- **HOMR/Oemer (Baseline):** Uses a dual U-Net (staff separator + symbol classifier) with heuristic filtering. It achieves high recall by identifying vertical lines spanning all staff lines as barlines, but some false positives slip through (e.g. noise blobs or tall stems) [1] [2]. It was pre-trained on CVC-MUSCIMA (handwritten) and DeepScores (synthetic) [3]. Licensing is mixed (Oemer MIT, HOMR AGPL), and it runs on an RTX 4060 with moderate effort to fine-tune [4].

- **YOLO-based Detectors:** Treat barlines (and other symbols) as objects for single-stage detection. They showed **high precision on clear barlines**, with the ability to adjust confidence thresholds to prune false positives [5]. However, distinguishing tall note stems remains challenging without additional context [6]. YOLO models (e.g. Ultralytics YOLOv8) have pretrained weights available and would require fine-tuning on a few thousand score images for optimal results [7]. Notably, Ultralytics' code is GPL, but the survey suggested reimplementing in an Apache-licensed framework (e.g. Detectron2 or YOLOX) to avoid license issues [8] [9].

- **Mask R-CNN Instance Segmentation:** Provides pixel-level masks for symbols, which helps **differentiate barlines from stems by their full-staff-length shape** [10]. A well-trained Mask R-CNN can output one continuous mask per barline, even if the barline intersects other symbols [11]. This approach significantly reduces stem-vs-barline confusion because a predicted mask spanning ~5 staff spaces with no notehead attached is classified as a barline, whereas a shorter mask terminating at a notehead is classified as a stem [10]. The trade-off is complexity: Mask R-CNN is heavier and slower than YOLO (two-stage detection + segmentation) [12]. In prior experiments, training on ~1k images (e.g. printed **DoReMi** or **DeepScores** data) yielded about **60% mAP** on dense score symbol detection [13] – indicating good potential but also room for improvement. Pretrained Mask R-CNN models on COCO are a starting point; additional domain pre-training on MUSCIMA++ (handwritten, barline masks available) can boost accuracy [14]. The implementation can leverage Detectron2 (Apache-2.0) with no licensing barriers [15].

- **DETR + Notation Graph:** A transformer-based detector (as in a 2021 thesis by Allen) would use global context and **learn symbolic relationships to filter false barlines** [16] [17]. For example, DETR could learn that a vertical line with no attached noteheads is likely a barline, whereas a line connected to a notehead is a stem [16]. It can even handle multi-staff barlines by linking barline segments across staves in its output graph [18]. This approach inherently organizes symbols into measures, potentially outputting measure separator classes if trained with such annotations (MUSCIMA++ has a "measureSeparator" class) [19]. However, DETR is experimental in OMR: it requires **extensive relational training data** (MUSCIMA++ graph annotations, etc.) [20], is memory-intensive on high-res images [21], and would be a significant R&D effort for our project [22] [23]. License-wise it's fine (Facebook DETR is Apache/MIT) [24], but it was deemed **overkill for a quick win**, better suited for long-term exploration [25].

- **End-to-End Seq2Seq OMR:** These models (CNN+Transformer or CRNN encoders with sequence decoders) output a full music transcription including barline tokens. They **tend to insert barlines only where musically plausible**, so false barlines are rare. Measure numbering is inherently solved since the output sequence or MusicXML contains the measures in order [26] [27]. However, such models might **miss faint or unusual barlines** if they weren't learned well, and overall accuracy is tied to general transcription fidelity [28] [29]. Training these end-to-end networks requires large annotated corpora (images to symbolic) and significant compute. While promising for consistency (e.g. ensuring rhythmic validity across measures), they are not easily "plugged in" to our pipeline for just the barline task. Moreover, many are research prototypes without readily available pretrained weights. Given our goal of an incremental improvement, the survey suggested focusing on object-level detectors (YOLO/Mask R-CNN) first, and treating seq-to-seq OMR as a longer-term direction.

In summary, the prior analysis indicated that our best short-term path is to incorporate a trained **barline-specific detector** (like a YOLO or Mask R-CNN model) using existing OMR datasets. This could substantially reduce false positives beyond HOMR's heuristics while maintaining high recall. The survey also stressed using **public data** (e.g. DeepScores, MUSCIMA++, DoReMi) for training instead of manual annotation, and being mindful of licenses – e.g. avoid GPL code or data that would complicate future commercial use [8] [9].

Dataset Survey

Below is a survey of public datasets relevant to detecting barlines, staff lines, or measure regions in sheet music. For each, I note the data format, annotation of barlines, size/domain, license, and pros/cons for our use case (improving barline precision while keeping recall high):

DeepScores V2 (Synthetic Printed) [30] [31]

- **Description:** A large **synthetic** dataset of densely engraved printed music, generated from MusicXML via LilyPond in multiple fonts [32] [33]. It offers **255,385 full-page images** (with a smaller "dense" subset of 1,714 images representing the most complex pages) [34] [35]. Annotations cover **135 symbol classes** (notes, clefs, rests, **barlines**, etc.) with multiple formats: non-oriented and oriented bounding boxes, as well as pixel-level semantic and instance segmentation masks [36].

- **Barline Annotations:** Barlines are explicitly labeled as an object class. All standard barline types that appear in notation are included (single, double, repeats, etc.) – either as distinct classes or as barline objects plus separate repeat-dot symbols. We can thus directly train a detector to recognize barlines. If needed, repeat dots could be post-processed to refine barline types (e.g. if a barline bbox contains adjacent repeat dot objects).

- **Size & Domain:** Extremely large (over 250k images, 151 million symbol instances) [37], though the authors suggest the 1.7k "dense" subset is often sufficient for research [35]. All images are **computer-engraved printed music** (no handwriting). They are high-resolution (around 2475×3504 at 300 DPI) and clean – no skew, noise, or broken ink, though random affine distortions and five different music fonts were used to add some variety [38] [39]. Domain-wise, this matches our printed scores, except DeepScores images are ideal renderings (binary or grayscale) without real scan artifacts.

- **License: Creative Commons Attribution 4.0 (CC BY 4.0)** [31] [40] – very permissive, allows commercial use as long as we credit the source. This is a strong positive for our purposes.

- **Pros:** Massive scale ensures a wealth of barline examples in varied contexts. Because it's synthetic, **ground truth is perfectly accurate**. Barlines appear with all manner of surrounding symbols (high density), providing lots of hard negative examples (stems, thin lines) to learn from. It includes instance segmentation masks, enabling advanced approaches like Mask R-CNN if desired. The **license is compatible** with commercial use, so we can train on it and use resulting models freely. Also, pretrained models on DeepScores exist (from prior work) which we might leverage.

- **Cons:** The gap between synthetic and real scanned images is a concern. Models trained purely on DeepScores might be prone to false positives on scanning artifacts or faint noise (which never occur in the synthetic data). Also, DeepScores barline annotations are pixel-perfect straight lines; a model might not immediately generalize to slightly curved or broken lines in real scans. We'd likely need to apply augmentations (e.g. add noise, distortions, thicker/thinner lines) to simulate scan conditions [41]. Additionally, the dataset's completeness (every barline labeled) could lead a model to assume **no missing barlines**, whereas in real life a weakly printed barline might drop out – but since our baseline had zero false negatives on page_3, maintaining recall is less of a worry. Lastly, using the full dataset for training is heavy; we might start with the 1.7k "dense" images subset for practicality.

MUSCIMA++ (Handwritten) – with Measure Annotations [42] [43]

- **Description: MUSCIMA++** is a dataset of **140 handwritten music pages** derived from the CVC-MUSCIMA set [44]. It provides **pixel-level annotations for every musical symbol**, plus a graph of relationships (e.g. which notehead is attached to which stem) [44]. Notably, it includes barlines and allows linking barline segments across staves as part of the notation graph. An extension by Pacha ("MUSCIMA++ Measure Annotations") reformatted a subset of these labels specifically for **measure and staff recognition tasks**, including ground-truth barline locations in JSON and COCO formats [42]. In total, ~140 images with barline annotations are available.

- **Barline Annotations: Explicit and pixel-precise.** Each barline is labeled as a separate object mask (covering the exact pixels of the barline). This includes partial barlines in multi-staff systems (e.g. if a score has separate barlines per staff, each is annotated). Repeat barlines would have the barline itself plus separate symbols for the dots. Additionally, MUSCIMA++'s notation graph marks certain barlines as "measure separators" and links repeat dots to barlines, which could be leveraged in a relational model [19].

- **Size & Domain:** Small (only 140 pages, each with ~20-30 measures). Domain is **handwritten music** (different styles of handwriting, since CVC-MUSCIMA had 50 writers). The barlines in these images are often slightly wavy or broken and might have gaps where staff lines were erased. Our use case is primarily printed music, so this is a **domain mismatch**. However, the variety of handwriting might be useful to make a model robust to imperfect lines or partial barlines.

- **License: CC BY-NC-SA 4.0** (Creative Commons Non-Commercial) [45] [46]. This is problematic for future commercial use – we could **not** freely use a model trained on MUSCIMA++ in a commercial product without permission. Even though it's publicly downloadable, the non-commercial clause is a red flag. We might use it for experimentation or pre-training, but relying on it would pose a licensing risk.

- **Pros:** The **pixel-level accuracy** of barline masks can teach a model fine discrimination (e.g. ignoring stray ink that doesn't form a full barline). It's one of the few datasets with **measure-level semantics** – e.g. it explicitly labels which barlines separate measures, which could help if we ever incorporate measure grouping in detection [19]. Training on MUSCIMA++ could imbue a model with some **robustness to imperfections** (since handwritten barlines can be discontinuous or curved). Also, it's easily available in COCO format for quick experiments [42].

- **Cons:** The small size means it's not sufficient by itself to train a high-capacity model – overfitting would be a risk. More importantly, the **non-commercial license** makes it risky for our end goal. We likely cannot incorporate this data into a final product without violating terms. Thus, while MUSCIMA++ is valuable for academic experiments (and possibly as a validation set to test if a model generalizes to handwriting), it's not ideal for our production pipeline. Any model we seriously consider deploying should rely on permissively-licensed data.

DoReMi (Synthetic Printed) [47]

- **Description: DoReMi** is a newer synthetic dataset (2021) of printed music generated with Steinberg's Dorico notation software [48]. It contains **6,432 images** of scores (mostly one system per page), with nearly **1 million annotated symbols** spanning **94 classes** [49]. Alongside the images (which can be binary or color), it provides MusicXML and a rich XML metadata for each page, including bounding boxes and pixel masks for each symbol, plus high-level musical information (e.g. note durations, pitches, and crucially the linkages between noteheads, beams, stems, slurs, and so on) [48] [50]. It essentially marries the approach of DeepScores (comprehensive symbol labels) with some semantic richness like MUSCIMA++ (explicit relations between symbols).

- **Barline Annotations:** Barlines are explicitly annotated as symbols with bounding boxes and pixel masks. Because Dorico can export structured data, the dataset likely labels barline types distinctly or provides attributes (e.g. a barline that has repeat dots might either be a separate class or have "repeat" markers linked via the Outlinks field in the XML). There is mention of linking notation primitives; for example, repeat dots might be linked to the barline in the metadata [48]. Even if not, we can treat all barlines as one class for detection, then use the presence of repeat dot symbols (also labeled) to post-classify repeats.

- **Size & Domain:** Moderately large (6.4k images is substantial, though smaller than DeepScores). The images are printed and **synthetic**, but Dorico's engraving is very high-quality and closer to real publisher print than LilyPond's default look. Dorico can also produce various engraving styles (it wasn't stated how much variation in fonts/appearance is in DoReMi, but presumably some). The dataset likely covers a wide range of notational elements (94 classes means more symbol types than DeepScores) and possibly a variety of musical genres or layouts. The images might be cleaner than scans but could be exported in greyscale or even with slight "aging" effects – this isn't explicitly stated, but the mention of color images suggests they might include realistic elements or simply color notation (e.g. for highlighting). In any case, domain is **synthetic printed** like DeepScores, meaning we still should account for differences to real scans.

- **License: Unclear.** DoReMi is hosted on a Steinberg Media GitHub, but no explicit license is noted [51]. The OMR dataset index doesn't list a license for it [52], implying one might need to agree to terms or it's meant for research use. Given Steinberg's involvement, it might be available for academic use freely (with required citation) but not clearly permitted for commercial use. This uncertainty is a **potential issue** – we'd need to investigate if Steinberg allows using DoReMi-

trained models in commercial products. Without a clear license, it's safer to assume it's not freely commercial-friendly.

- **Pros:** Content-wise, DoReMi is very relevant. It's **focused on printed scores** and has a **diverse symbol set**, likely including barlines, repeats, double bars, etc., in various musical contexts. The additional metadata (MusicXML, relationships) means it could support advanced training (like a DETR that learns relations, if we ever go that route). But even for plain detection, having another source of synthetic images can help a model generalize (since Dorico's output will differ slightly from LilyPond's). The dataset's size is manageable – using a few thousand images from it for training is feasible on our hardware.

- **Cons:** The main concern is the **license/availability**. If we cannot confidently use it for commercial outcomes, we should be cautious. Another practical con: since it was generated by Dorico, it might not intentionally include as much noise or variability as needed for robust real-world performance (though this is true of any synthetic data). It also might have mostly one system per page – so things like multi-system page detection or partial barlines connecting staves may not appear as often as in a full score. If our use case includes full orchestral scores with system dividers, DeepScores might have more of those examples than DoReMi.

(Aside: There is an overlap between DeepScores and DoReMi – the DoReMi paper intended it to complement DeepScores and MUSCIMA++ [53]. In practice, we could combine DeepScores and DoReMi to get a broader training set, if licensing can be sorted out.)

AudioLabs Measure Annotations (Real Scans, Mixed Sources) [54] [55]

- **Description:** A unique dataset from an audio-score alignment project (ISMIR 2019 by Frank Zalkow et al.) provides **measure-level annotations on real sheet music scans** [54]. Specifically, it includes several hundred pages of classical music (e.g. the entirety of Wagner's Ring cycle, Beethoven piano sonatas, Schubert's Winterreise, and some choir pieces) with ground-truth **bounding boxes for each measure** on each page [56]. The images are actual digitized scores (some likely public domain scans, others possibly provided by a publisher). A measure bounding box essentially spans from one barline to the next across the system. The dataset includes a Python notebook demonstrating how to parse and display these measure boxes [57].

- **Barline Annotations:** Not directly labeled as barline objects, but **implied** by the measure boundaries. If we have bounding boxes for measures, the vertical lines at the box edges correspond to barlines (or the page margins for the first/last measures). Also, the dataset distinguishes system measures vs. stave measures (with over 24k system measure boxes annotated and 11k stave-level annotations in an expanded version) [58]. This implies that for multi-staff systems, the measure box covers the full system, and there may be additional info to link staff-level barline segments. We could derive barline coordinates by taking the left/right edges of these measure boxes on each staff.

- **Size & Domain:** On the order of **hundreds of pages** (the Ring cycle alone is ~250 pages, plus others). Domain is **real scanned printed music** – likely high-quality scans, but there could be typical issues (scans of older prints might have slight noise, skew, or thickening/thinning of lines). These are primarily 19th-century notated music (romantic opera, classical sonatas, etc.), so lots of standard barlines, repeats, etc., in real engraving styles. Because it includes entire pieces, it covers scenarios like multi-page rests (double barlines), final double bars, etc. It's excellent real data for measures, though not as diverse in symbol classes as the synthetic sets.

- **License:** The OMR dataset index lists this as **"License: Mixed"** [55] . This likely means some images are from public domain sources (okay to use), while others (especially those "selected pieces from Carus Verlag") are under copyright and used with permission for research demos [59] [60] . Indeed, the acknowledgement thanks Carus for providing scores [61] . So, while the measure annotations are provided openly (and the download is public) [62] , using all of the images for commercial purposes is questionable. We might separate out the obviously public-domain ones (e.g. Wagner, Beethoven, Schubert should be PD) from any modern editions that might be protected. Nonetheless, it's not a clean permissive license across the board.

- **Pros: Real scanned data** – this directly tests and improves a model's robustness on actual conditions. A model fine-tuned on these could learn to ignore common scanning artifacts and handle real engraving line thickness. The measure-level annotation is exactly aligned with our ultimate goal: if a model can detect measures (or barlines) in these pages, it's solving measure segmentation in a real scenario. We could use this to validate our barline detectors (e.g. does it find the same measure splits?). Also, it's a relatively large set of full-page images, which is great for evaluation even if we avoid training on it for license reasons.

- **Cons:** Since barlines aren't individually labeled, training a model on measure boxes would be indirect – we'd have to convert measure-box data into barline targets (essentially the vertical line at each box edge). That conversion might be lossy if the barline is not perfectly vertical (the box edge might cut through a slanted line in an old scan). Also, measure boxes don't distinguish between single vs double barlines; the model would just see the region end. As a training set, it's also somewhat limited in notation diversity (mostly common practice era notation). Finally, the **mixed licensing** means we'd need to vet which pages are safe to use. It might be safest to use this dataset primarily for **evaluation** of our models on real scans, or as a light fine-tuning dataset (on the clearly PD portions) if needed.

After considering many datasets, the **most relevant ones for barline detection** appear to be **DeepScores** (for a huge volume of printed training data with clear barline labels) and **DoReMi** (for additional printed data, though with license caveats). MUSCIMA++ is valuable for concept and possibly as a small supplement, but risky license-wise. The AudioLabs measure data is very relevant to our task and could be used for testing and perhaps selective training, but we'd treat it carefully due to licensing.

In summary, we have at least **two permissively-licensed datasets** ideal for training a barline detector on printed music: **DeepScores V2 (CC BY 4.0)** and the **dense subset of it**, and possibly the **public domain portion of the AudioLabs measure set** (if we extract those). **DoReMi** is promising if we clarify its terms, and **MUSCIMA++** could be used in a non-commercial research context but not in a final product without permission.

Pretrained Model / Repository Survey

Next, I surveyed repositories and pre-trained models that could be leveraged for barline detection. The focus is on projects that have **ready-to-use code and weights** for musical symbol detection or similar tasks (staff detection, measure recognition), ideally trained on the datasets above or comparable data. For each, I describe the approach, architecture, output, weight availability, license, and how applicable it is as a drop-in barline detector in our pipeline.

1. OMR-DLN (Deep Learning for OMR) by dmgs (GitHub) [63] [64]

- **Repo & URL:** "dmgonzalez8/OMR" on GitHub [63]. This project (likely a thesis or research project code) focuses on object detection for musical symbols using DeepScoresV2.

- **Task & Architecture:** It implements both **YOLOv8** and **Faster R-CNN** for symbol detection and even has a component for **measure detection**. Specifically:

- It trained **YOLOv8** models for symbol classification (detecting multiple classes including barlines) and a separate YOLO for measure regions [65].
- It also trained a **Faster R-CNN** (a two-stage detector like Mask R-CNN but without masks) on the same data for comparison [66] [67].
- The YOLO models leveraged Ultralytics' pretrained weights (on COCO) for initialization [68]. They tried YOLOv8n, m, x variants (small to large) [65].

- The measure detection YOLOv8 model treats each measure as an object (presumably by using the ground-truth barline positions to define measure spans) [69] [65].

- **Outputs:** For symbol detection, the YOLO models output bounding boxes with a class label among 94 DeepScores classes (barlines being one). For measure detection, the YOLO outputs bounding boxes that ideally span entire measures (so a detected measure-box implicitly gives the barline positions at its edges). The Faster R-CNN outputs bounding boxes and class labels similarly. No segmentation masks here (unless one were to extend it to Mask R-CNN).

- **Pretrained Weights: Yes – available via Google Drive.** The repo README links to a Drive folder with "latest models" for download [64]. Although I couldn't retrieve the listing here, it likely contains the weights for the trained YOLOv8 models and perhaps the Faster R-CNN model. The README also shares performance metrics: the best YOLO (YOLOv8x) achieved **$\mathbf{mAP_{50} \approx 0.728}$** across all symbol classes after 500 epochs [70], which is quite good given the difficulty of Dense scores. Barlines specifically should be among the higher-precision classes, as the confusion matrix suggests the model learned most classes well [71] [72]. The measure-detection YOLO was trained for 500 epochs (on YOLOv8m) – details of its accuracy aren't explicitly quoted, but presumably it can detect measure boundaries reliably in synthetic data.

- **License:** The repository does **not list an explicit license**. This is important: no license means all rights reserved by default, which is not ideal for our use. The code is based on Ultralytics YOLO (GPL) and perhaps some MMDetection, but without a clear open-source license from the author, using it directly in a commercial project would be risky. However, we might not need to use his code – just the concepts or pretrained weights. The weights, being data, could potentially be used if we treat them like a model checkpoint (though formally, the weights might be considered a derivative of YOLOv8's training, which is tricky under GPL). To be safe, we'd likely reimplement or re-train using our own framework (to ensure Apache or MIT code) rather than incorporate this repo's code.

- **Applicability:** Highly applicable. This gives us a **head start with YOLO on DeepScores**:

- We could take the **YOLOv8 model weights** trained on DeepScores and directly test them on our images. They should detect barlines (and many other symbols). We can filter for the barline class output to get barline predictions. We might need to merge or discard multiple overlapping outputs if it detects each staff's barline separately.

- Because it's trained on synthetic data, we expect **high recall** for clear barlines and likely decent precision, but some false positives on things like long rests or vertical artifacts might appear. We can calibrate the confidence threshold or fine-tune further.
- The **Faster R-CNN** model from this repo could be an alternative if we prefer two-stage detection. Faster R-CNN might be slightly more precise (slower, though) and we could extend it to Mask R-CNN if we have segmentation masks.
- Running these on an RTX 4060 is feasible: YOLOv8x is heavy but still real-time on GPU for one image, YOLOv8n/m are very fast; Faster R-CNN (ResNet-50 backbone likely) can also run at a few FPS on a page. Fine-tuning either on a few real images or additional data is possible on our hardware (the repo indicates training 500–1000 epochs took on the order of hours to tens of hours on presumably a similar GPU [73] [74]).

- One consideration: YOLOv8 is GPL for the code, but we could use an alternative engine (like exporting the model to ONNX or using YOLOX) to deploy it without GPL code. Also, the **Ultralytics license issue**: since they used Ultralytics' code to train, the weights are somewhat tainted by GPL. However, an argument could be made that model weights are not code – but it's a gray area. For an experiment branch, using it is fine; for product, we'd likely retrain with an Apache implementation. The repo even suggests possibly using Detectron2 or an MIT-licensed YOLO version to avoid this [8].

- **Likelihood of outperforming baseline:** Qualitatively, yes, especially in precision. A YOLO model trained on DeepScores should be **much better at filtering stems vs barlines** than the heuristic alone, since it has learned the visual differences in context. The prior survey noted YOLO's high precision on clear barlines [5]. I expect fewer false positives than HOMR, as long as the images aren't too far off domain. It should maintain recall, catching all real barlines unless something like a very faded barline is present (which synthetic training wouldn't cover well). We may need to watch for **false negatives if a barline is broken** or if the model mistakenly thinks a very light barline is not there – but with our baseline having 0 FN, we can't afford new misses. Fine-tuning on a bit of real data (even 20–30 real images with barlines) could alleviate that.

- **Practicality:** Very practical. We can fetch the YOLOv8 weights and use the Ultralytics `yolo` Python package to test on images in minutes. For integration, we could run the YOLO model as a separate step in our pipeline: input an image, get barline bounding boxes. Since it's not end-to-end, we'd then feed those detections into our measure assembly logic (counting barlines per staff). The YOLO model can definitely run on a single RTX 4060 (the author did so). If we fine-tune, the dataset is large but we can train on subset or use slicing strategies (as another repo did) to fit GPU memory [75]. In short, this is probably the **quickest path to a functioning alternate barline detector**.

2. MusicScanner (DeepScores YOLO) by CatUnderTheLeaf [76] [77]

- **Repo & URL:** "CatUnderTheLeaf/musicScanner" on GitHub [78]. This appears to be a small project (possibly a demo with Streamlit) that also uses DeepScoresV2 to train a YOLO model.

- **Task & Architecture:** It focuses on **object detection** for all symbols using a YOLO-family model. Notably it mentions using "YOLOv10" – which likely refers to the developer's internal name (perhaps YOLOv8 or YOLO-NAS). The approach included **slicing large images into 640x640 patches** for training due to hardware constraints [77]. It also used augmentation tailored to music (no random flips, slight rotation/scale, etc.) [79] [80]. Essentially, it's another YOLO-based symbol detector on DeepScores, similar to the above but less extensive in documentation.

- **Outputs:** A YOLO model that outputs bounding boxes for the 135 classes of DeepScoresV2 (including barlines). The project likely aimed at a **demo app** (it has a Streamlit link) where you can upload an image and see detected symbols. So it's a general OMR object detector, not specifically tuned just for barlines.

- **Pretrained Weights:** The repo doesn't directly include weight files, but since it's demonstrating results, the author likely has a trained model. They report training for 163 epochs achieving **mAP_{50} = 0.74** (and mAP_{50-95} = 0.54) on DeepScores, which is comparable to the other project's results [81]. There are plots and images in the README indicating the model's performance [81]. If needed, one could reach out or check if the Streamlit app exposes a way to fetch the model. Otherwise, replicating the training is possible (the README has details on data prep and augmentation). The code is Apache-2.0 licensed [82], so we could reuse it freely.

- **License: Apache-2.0** for the code [82]. This is a big plus – it explicitly sidesteps the Ultralytics GPL issue (maybe they used YOLOv5/7's open weights but reimplemented something? Or perhaps YOLOv5 which was also GPL… unless they only share their code which doesn't include YOLO core? It's a bit unclear, but the repo claims Apache license). Regardless, no portion of it seems encumbered. The DeepScores data it uses is CC BY, so fine.

- **Applicability:** This is quite similar in applicability to the first (dmgs) YOLO model: a generic symbol detector that we can use to get barlines. Since it's also DeepScores-based, it will have the same strengths (lots of training on barlines) and weaknesses (no real noise in training). Having two independent implementations is nice for cross-checking. The license being Apache means we could integrate this code or model more directly without concern. However, we need the actual weights to avoid retraining. If those aren't provided, we might prefer using the dmgs weights or just retraining ourselves using this code as a base (since 163 epochs to reach ~0.74 mAP suggests it doesn't take too long with slicing).

- **Performance vs Baseline:** Similar expectations as above – a well-trained YOLO should drastically cut false positives. Given the reported mAP and precision, barlines likely have very high precision in these models (the DeepScores objects like clefs, rests achieved >99% precision in dmgs's YOLOv8x [83], and barlines are usually easier to distinguish than some noteheads). So I'd expect this model, when run on our test page, might reduce those 30 false positives to just a handful (if any) at a reasonable confidence setting, while still catching all 152 real barlines. The risk of missing barlines is low unless a barline is severely faint or broken.

- **Practicality:** Very practical. The code uses slicing of images into tiles for training due to memory – but for **inference** on a single page, we could either tile or just run the model on the full image if our GPU can handle it (since YOLO can run on the full image if we use the right size model and enough VRAM; a 3000×2000 image might need to be resized or tiling if using a large YOLO). The repository's use of SAHI (slicing library) indicates that at least for training they needed it [84], but at inference, one can similarly tile or just slide a window over the image. This adds complexity, but it's solvable – or we retrain a smaller model that can take the whole image. On an RTX 4060 (8GB), a YOLOv8n or YOLOX might handle ~1500×1500 images at once; for full 300dpi pages, tiling is a fine approach. Code-wise, since it's Python and uses standard libraries, integration in our pipeline (which already uses Python/C++) can be done via a subprocess or a direct call. The Apache license means we could even incorporate parts of it into our code if needed.

- **Note:** This model wasn't specifically tuned for barline-only detection; it detects everything. Running it will yield lots of symbols. That's fine – we can ignore non-barline outputs. It is slightly

wasteful computation if we only care about barlines, but the model can handle it. If needed, we could even retrain it only on barlines (as a single-class detector) which might simplify things and speed it up further. The existing weights though likely benefit from multi-class context (the model learns to ignore noteheads attached to stems, etc., which helps indirectly in classifying barlines).

3. Mask R-CNN (Detectron2) – no single repo, but established approach

- **Repo & URL:** No specific official repo with pretrained OMR Mask R-CNN was found, but **Detectron2** (by Facebook) is an Apache-2 licensed library that provides Mask R-CNN out-of-the-box [85]. Some academic code (like the thesis by S. Allen on transformer OMR) might contain Mask R-CNN baselines, but none with readily packaged weights. Instead, one would likely use **Detectron2 with a COCO-pretrained backbone** and fine-tune on an OMR dataset. Since this task specifically asked for pretrained models, I include Mask R-CNN here as a viable approach which we could initialize with existing weights (COCO or even better, a model that someone trained on MUSCIMA or DeepScores if we find one).

- **Task & Architecture: Mask R-CNN** is a two-stage object detector that, in addition to bounding boxes, produces a pixel segmentation mask for each object. For our use, we would configure it to detect **barlines** (either as a single class or possibly multiple classes if we differentiate repeats, etc.). The architecture would likely use a ResNet-50 or ResNeSt backbone with an FPN (Feature Pyramid Network) – standard Detectron2 config – and then a small mask head to predict the barline shape [12].

- **Outputs:** Each detected barline comes with:

- A bounding box,
- A predicted mask (which outlines the exact shape of the barline),
- A class label (just "barline" in a single-class setup, or possibly "barline", "repeat barline", etc. if multi-class).

This means the model doesn't just tell us where a barline is, but also which pixels belong to it, which is useful if we need precise positioning (it can help differentiate two close vertical lines). In practice, for measure numbering we mostly need the line's position; the mask could allow us to determine its exact endpoints or continuity.

- **Pretrained Weights: Yes (partially)** – Detectron2 models pretrained on COCO are widely available (Mask R-CNN with ResNet-50). That gives a good initialization, though COCO doesn't have barlines. More relevantly, there was an experiment in literature where Mask R-CNN was trained on a combination of printed score data (like DeepScores or a subset thereof). The prior survey mentioned **MUSCIMA++ has barline masks and DeepScores provides synthetic masks** [41]. If we find weights from an academic paper: e.g., an ICDAR 2019 paper by Calvo or Baró might have done something, but none were openly published as far as I know. So likely we'd fine-tune ourselves on available data. In summary: we have readily pretrained backbones, but not an off-the-shelf **barline-trained** Mask R-CNN model to download. We'd have to train it (which is a moderate effort, but doable with 1k–2k images and an RTX 4060 in perhaps a few hours of training).

- **License: Apache-2.0** (for Detectron2 and typical model zoo weights). Any dataset we train on must be permissible (so we'd use DeepScores which is CC BY, making the resulting model fine for

commercial use). So from a licensing perspective, a Mask R-CNN we train on DeepScores or other permissive data is completely safe to use. This approach avoids the GPL issue entirely.

- **Applicability as drop-in:** A Mask R-CNN detector could slot into our pipeline as a more precise barline finder. For example, we could run it on the full page image (or potentially on staff-cropped images if we want to simplify the task). It would output barline masks; we then convert those masks to lines or bounding boxes and integrate that with staff info. Because Mask R-CNN can learn the concept of a continuous barline, it might reduce false positives in tricky cases (e.g., if a stem is very close to a staff line, HOMR might misclassify it as a barline; Mask R-CNN would likely **not** output a full vertical mask there because the shape doesn't match a barline). The prior survey noted that with enough training, Mask R-CNN becomes "resistant to confusion" between stems and barlines due to mask shape and context [86] [10].

- **Expected performance:** Likely **very high precision**, potentially even better than YOLO's bounding boxes, because the model learns a shape prior (long vertical line). It essentially performs a bit of what our heuristic did (checking length) but in a learned way. For recall, Mask R-CNN should catch nearly all real barlines as well; it might even detect broken barline segments and still classify them as barline if enough of the shape is there (stitching them via the mask). The survey cited ~60% mAP on dense symbol detection [13], but that was across all classes; for the barline class specifically, I'd expect higher AP because barlines are relatively salient. Also, training data quality matters: DeepScores synthetic masks are perfect but may not simulate broken lines; MUSCIMA++ has broken ones but is small. We could augment training with some erosion to simulate gaps.

- **Practicality:** The downside is **we have to train or fine-tune it ourselves**. But given we have data, that's not from scratch labeling, just running training code. On an RTX 4060, training Mask R-CNN on ~2k images (like the DeepScores dense set plus some augmentation) could take perhaps a day or two to reach top performance, which is acceptable for an experiment branch. Inference speed is slower than YOLO – maybe 1 image per second at high resolution – but that's still fine for our use (we're not doing real-time video, just processing pages). Integration-wise, Detectron2 outputs could be read and then we'd map mask coordinates to staff indices (though if needed, we can simplify by projecting the masks to vertical lines and find their x positions). A possible integration approach: run Mask R-CNN after a faster model's proposals – e.g., use HOMR or YOLO to propose barline regions, then Mask R-CNN to confirm/refine them [87] [88]. But even standalone, it can replace HOMR's barline stage.

- **Risks/Caveats:** If we rely only on synthetic training, the Mask R-CNN might be overkill if it doesn't generalize – i.e., we might get fewer false positives but potentially a few misses if a real barline doesn't exactly look like the synthetic ones (e.g., very thick or doubled by printing). Also, the output is more complex (masks). But as noted, Detectron2 is well-supported, and we have full control to adjust it. Another risk is the time investment: YOLO is easier to get running quickly; Mask R-CNN might require more hyperparameter tuning (learning rate, etc.) to get that extra precision. Nonetheless, it's a prime candidate since it's known to excel at this kind of shape-based detection [11] [89].

4. Audiveris (classical OMR tool) – for completeness

- **Repo & URL:** Audiveris is a long-standing open-source OMR software (on GitHub, audiveris/audiveris). It's not a single model but a whole pipeline. I mention it only to consider if it has any model we can reuse:

- It does staff line removal via image processing, then runs symbol recognition (some of which uses neural networks for classifier of connected components).
- It does detect barlines, but primarily with rule-based methods (looking at vertical connected components crossing staves).

- Pretrained "models" in Audiveris are small CNNs for classifying shapes, not object detectors.

- **Output & License:** It would give us directly measure-separated MusicXML. But Audiveris is AGPL licensed, which is risky for us. And integrating it partially is non-trivial.

- **Applicability:** Given we already have HOMR which is similar in concept but newer, Audiveris is unlikely to outperform our current baseline on barlines. Audiveris often suffers similar false positives on noise, etc., unless carefully tuned. Therefore, I would not prioritize it as a "model to integrate" – it's more of an alternate pipeline. It's mentioned here only as a known tool. I would prefer the ML models above over trying to incorporate Audiveris.

Summary of model options: The most promising pretrained or easily-trainable models for barlines are the **YOLO-based detectors** on printed music and the potential of a **Mask R-CNN model** on the same. Repositories like dmgs/OMR and musicScanner show that community has achieved good results with YOLO on DeepScores. While we have to be cautious about licenses (avoid directly using GPL code in production), we can use those weights or retrain with Apache implementations (like YOLOX or Detectron2). Both YOLO and Mask R-CNN approaches can run on our hardware and integrate into a multi-stage pipeline (just feeding images and getting barline predictions).

The YOLO route is more **ready-made** – we could likely plug in a pretrained YOLO model and see immediate improvements. The Mask R-CNN route is more **fine-tunable** – might require a bit of training effort but could yield the ultimate precision in reducing FPs. DETR-based models are still experimental and not readily available off-the-shelf, so I would set them aside for now (no pretrained DETR for OMR is published yet, to my knowledge).

Thus, I will focus recommendations on combinations of the datasets and models described above, to harness the strengths of each.

Shortlist & Recommendations

Combining the findings from the dataset and model surveys, I propose the following **shortlist of 3 candidate combinations** (dataset + pretrained model or architecture) that appear most promising for improving barline detection in our `feature/barline_model_experiments`:

Candidate 1: DeepScores V2 + YOLO (Object Detector)

Justification: DeepScores provides an abundance of **printed barline examples** with a permissive license, and YOLO has demonstrated high precision on this task [5]. A YOLO model (e.g. YOLOv5/7/8 or YOLOX) can be trained or fine-tuned on DeepScores to explicitly detect barlines as objects. This pairing directly addresses our needs: DeepScores' scale helps ensure **zero missed barlines (high recall)**, and YOLO's confidence tuning can **filter out spurious barline-like elements (fewer FPs)** [90]. We already have evidence (from community projects) that YOLO on DeepScores can achieve >70% mAP and very high

precision [72], meaning it can likely outperform our heuristic baseline in false positive reduction. Also, using DeepScores avoids any licensing issues and gives a model applicable to all printed notation.

Integration Sketch: We have two paths: - Using a Pretrained Model: Acquire the YOLO weights from dmgs's OMR project or the musicScanner project. For example, download the YOLOv8 model trained on DeepScores (assuming we can get the `.pt` file from the author's Google Drive or by contacting them). Then use the Ultralytics YOLO Python API (or a converted ONNX) to run inference on a page image. This will output a list of detected bounding boxes with class labels and confidences. We filter for the "barline" class (DeepScores likely labels it as something like class id 40 or by name). The vertical positions of these boxes give the barline locations on each staff. We then take these detections and compare to ground truth: in our eval script, we'd treat each predicted box as a barline location (perhaps even just the x coordinate and span). We can compute IoU with true barline regions or simply check if the prediction overlaps a true barline – since barlines are thin, a tolerance on x-position and full y-span overlap could define a true positive. The evaluation would then yield TP/FP/FN counts. We expect FPs to drop significantly relative to HOMR. - Training our Own: Alternatively, use YOLOX (which is Apache-2.0 licensed) with a COCO-pretrained backbone. Convert a subset of DeepScores (maybe the 1,714 dense images, or more if needed) into YOLOX training format (COCO JSON or YOLO text). Then fine-tune for a single class "barline". This could be done in our environment; given YOLO's efficiency, a few hours on the RTX 4060 might suffice for a good model (the prior survey suggested a few thousand images and fine-tuning is feasible [7]). Once trained, integrate inference similarly as above. This avoids any GPL concern and we can tailor the model (e.g., include slight augmentations like adding Gaussian noise or distortion to simulate scans).

To use it in the pipeline, one approach is: **run the YOLO detector after staff detection**. Staff detection (from HOMR or elsewhere) can give staff bounding boxes; we could run YOLO either on the whole page or on each staff-zone to possibly reduce search area. But YOLO is fine on the full page too. The output barline boxes, we then group by staff (using x coordinate proximity and staff y-bounds) and then proceed with measure grouping across staves as HOMR did. Essentially, YOLO replaces the barline-finding part of HOMR. The rest (counting measures, numbering) stays the same.

Risks/Caveats: The biggest caveat is **domain mismatch**: a YOLO model trained on pristine images might trigger on noise in a scan. For instance, a smudge that spans through staff lines could confuse it. However, because YOLO also sees other classes, it might label a stray mark as something else (or with low confidence). We should be ready to fine-tune or calibrate. Another risk is the **license of YOLO code** – as noted, Ultralytics' trainer is GPL. But we can mitigate that by either using the weights in a non-GPL runtime (export to ONNX and run with an ONNX runtime, or use YOLOX which is Apache) [9]. The **size of the model** is a minor consideration: YOLOv8x is ~130 MB and quite heavy; we might opt for YOLOv8m or YOLOX-s which still perform well but are lighter, since we don't need to detect 100 classes at once. Lastly, YOLO's output might include separate detection for each staff's barline segment in a multi-staff system (or it might capture the entire connected barline as one box if it's a full system barline). We must handle that – but that's similar to how our ground truth is defined (likely one barline per staff, which is fine).

Candidate 2: DeepScores (with augmentation) + Mask R-CNN (Detectron2)

Justification: This pairing aims for **maximum precision** in barline detection by leveraging shape-based segmentation. DeepScores supplies a huge set of symbol **masks**, including barlines, which can train a Mask R-CNN model to recognize barlines by their pixel shape. Mask R-CNN (via Detectron2) has proven effective in distinguishing touching symbols and learning structural patterns [89] [10]. By training on DeepScores (and optionally a bit of MUSCIMA++ or augmentations to simulate noise), we get a model that

should **very accurately localize barlines and ignore stems**, since it learns that barlines are long vertical masks unbroken by noteheads [10]. The prior survey explicitly highlighted Mask R-CNN's strength in reducing stem confusion through its instance mask predictions [86]. Additionally, Detectron2 is robust and fully open-source (Apache), making integration and future use safe.

Integration Sketch: - Training: We would prepare a dataset for Detectron2. This could be the **DeepScores V2 dense set** with annotations filtered to just barlines (and possibly repeat dots if we want a second class). We can use the provided instance segmentation ground truth [91] to create COCO-format annotations for barline masks. If needed, augment these images by adding random noise, small affine distortions, maybe erase small vertical gaps to mimic broken lines [92]. Then, initialize a Mask R-CNN R50-C4 model with COCO weights (so it starts knowing edges/shapes generally) and fine-tune on this data. We might also include the **MUSCIMA++ barline masks** (140 images) as a small additional training set – but carefully, since that's CC BY-NC; perhaps use it just to validate or early-stop. - Inference: Using Detectron2's Python API, load the trained model and run `predictor(image)` for a given page. This will return a set of detected instances: each with a binary mask, a class, and a confidence. We filter to class "barline". We can then take each mask and compute its bounding box or its centroid line. For evaluation, likely converting masks to bounding boxes spanning the staff height is sufficient to compare with ground truth barlines (since ground truth might be in terms of a line segment). We then count TP/FP/FN as usual (IoU > some threshold or maybe mask IoU, but a true barline should overlap a predicted mask extensively). - Pipeline integration: We could run this on the whole page image; Mask R-CNN will detect all barlines in one go. If the page has multiple staves, the model should find each barline segment as a separate mask (unless two staves' barlines touch, which is rare except grand staff which usually has a single barline spanning both – in that case the model might output one mask covering the full connected barline; we can handle that by splitting by staff if needed using staff bounds). After detection, we assign each mask to a staff (based on overlap with staff areas) and then group vertically across staves if they align (for multi-staff measures). From there, measure counting is straightforward. Essentially, we replace the heuristic "span staff lines check" with the learned mask outputs and a simpler rule like "if a barline mask spans nearly the full staff height and isn't attached to a notehead mask, it's a barline" – which the model already enforces internally [10].

Risks/Caveats: - **Training effort:** This approach does require doing the training ourselves. If our timeline is tight, this is extra work compared to using an existing YOLO model. However, the benefit is a tailor-made model with likely superior performance on the specific distinction we care about (vertical lines). - **Generalization to scans:** We must ensure the model sees some "imperfect" examples during training or augmentation. Otherwise, it might expect perfectly straight, uninterrupted lines. If it encounters a slightly broken barline in a scan, it might output two smaller masks (and possibly not classify them as barline). We can mitigate this by: (a) adding artificial breaks or noise to some training barlines, (b) including a few real scanned pages (maybe from the AudioLabs PD set) in training just for realism. Even 20 real images with annotated barlines could help immensely. Since manual annotation is something we want to avoid, we might leverage the measure-box dataset: generate barline masks from those measure boxes for PD pages and include them in training. - **Inference speed:** Mask R-CNN is slower than YOLO. On a full 3500×2500 image, inference might take ~0.5 to 1 second per image on GPU. This is actually still fine for us (we're not doing bulk in real-time), but worth noting. If we had to process thousands of pages, it's still okay (a few hours at most). - **Multi-class or single-class:** We would probably train it as single-class for simplicity (barline vs background). It will then detect any type of barline. If we also wanted it to identify repeats or double bars, we could introduce classes, but that complicates training and might confuse it if data is limited for those sub-classes. Better to do single class and then post-process the mask: e.g., if repeat dots are detected by a separate process (or even by HOMR or by simple template matching), we can combine that info. The model, if trained only on barline masks, might ignore the repeat dots (since they're not part of the barline mask annotation). This could actually help – it will still mark the barline line itself as one continuous mask, and we won't get a split detection.

Why this combination: Ultimately, a DeepScores-trained Mask R-CNN could become a **drop-in replacement for Oemer's barline stage** with likely far fewer false positives. It leverages a strong open dataset and an open framework, aligning with our licensing needs. This addresses both goals: reduce FPs (by learned shape filtering) and keep recall (tons of training examples). It might require more work to implement than YOLO, which is why I'd label it as Candidate 2 (perhaps after trying YOLO). But it's an ideal backup if YOLO doesn't deliver the precision we want, or if we run into any YOLO license/ maintenance issues.

Candidate 3: AudioLabs Real Scans + Fine-Tuned Detector (Hybrid)

This is a bit more experimental, but I include it as a third option:

Justification: Use the **real scanned data with measure annotations** to fine-tune either of the above detectors, thereby specifically targeting false positives on real images. For example, we could take a YOLO or Mask R-CNN model pre-trained on synthetic data (DeepScores) and **fine-tune it on a few hundred real pages** from the AudioLabs dataset (excluding any proprietary ones). The rationale is that even a small fine-tune on real data can adapt the model's features to actual noise, lighting variations, etc. The measure annotations give us barline positions without needing manual labeling: we can derive barline ground-truth by drawing vertical lines at measure boundaries on those pages.

Integration Sketch: Say we start with the Candidate 1 model (DeepScores YOLO). We then prepare training data from AudioLabs: for each scanned page image, use its measure bounding boxes to mark barline locations (each box edge becomes a line – possibly multiple lines per system if internal repeats). We assign those as "barline" objects for training (with a small width but full staff height). Then fine-tune the YOLO model for a few epochs on this real data. This should adjust the model to better ignore things that in DeepScores never occurred (e.g., speckles or paper texture). After fine-tuning, we run inference as usual. Alternatively, do similarly with Mask R-CNN: generate pseudo-mask annotations for barlines on the scanned pages (basically straight line masks from top to bottom of staff at each ground-truth barline location). Then fine-tune the Mask R-CNN on those along with original data.

Pros: This combo directly addresses any **domain mismatch**. It uses an available dataset to teach the model "these are what real barlines look like, and here's what not to mark." It could further reduce FPs, e.g., if the model was confused by serif digits or text on the page, seeing them in fine-tuning with no barline labels there will teach it not to hallucinate barlines. It might also improve recall if, say, some barlines in real scans are thicker or broken – the fine-tune could help the model handle that (since measure boxes ensure those still count as barlines).

Cons: Licensing is the main issue. The AudioLabs set is mixed-license. We would include presumably PD music (Wagner, Beethoven, Schubert) which should be fine (those scans are likely from old editions, presumably PD images – but we should double-check source). The pieces from Carus Verlag likely cannot be used for anything but evaluation; we'd exclude those from training. So our fine-tune set might be smaller (maybe 200-300 pages that are definitely PD). This might be enough, though. Also, the measure boxes are not as precise as our other data – if there's slight misalignment, we might introduce a tiny label noise (e.g., the box edge might cut slightly into a note if layout is uneven). But on the whole, a measure box is a reliable indicator of a barline on each staff.

Applicability: This is not a standalone model; it's **complementary** to the above. I'd use this strategy if we find the synthetic-trained model has some trouble on our actual scanned pages (perhaps raising false positives or minor alignment issues). It's essentially a domain-adaptation step. It's practical: we already have the data and a starting model, fine-tuning on a few hundred pages is maybe an hour or two of

training for YOLO, or a bit more for Mask R-CNN. Running the fine-tuned model is the same as before. So, this candidate is really a refinement to Candidate 1 or 2 rather than an independent approach.

Risks: We have to ensure we don't accidentally include non-public-domain images in training (to keep things legally clean). Also, if the measure annotations have errors (unlikely, but if the semi-automatic tool missed something), we could be training on slightly incorrect labels. However, given the quality of that project, it's probably very accurate. Overfitting to that set could be a concern – but since it's diverse music and we'd do a light fine-tune, it should be okay.

After evaluating these, I will make the following recommendations:

Primary Candidate to Try First: DeepScores + YOLO detection (Candidate 1)

I recommend starting with the DeepScores-trained YOLO model. It's the **fastest path to see improvement**: we could likely get a model running within a day and test it on our page_3. YOLO's simplicity and the availability of pretrained weights make it easy to integrate. This model should immediately show whether we can knock out the false positives. Given the evidence, I expect it will drastically reduce false barlines (the confidence threshold can be set such that only very clear full lines are detected) [90] . It will also be straightforward to measure: we plug it in, run detection, and compute the new TP/FP metrics. Another reason to start here is that if something goes wrong (say the model misses some barlines due to a quirk), it's quick to adjust (retrain or tweak threshold). We also circumvent licensing issues early by either using YOLOX or handling Ultralytics code carefully – but since this is an experiment branch, we could even use the Ultralytics library for now to gauge performance, then later implement a non-GPL inference if it's promising. Overall, this candidate is **low implementation effort, high expected gain**, and aligns with the suggestion from our prior survey to try YOLO first as a baseline improvement [93] .

Backup Option: DeepScores + Mask R-CNN (Candidate 2), with possible fine-tune (Candidate 3)

As a backup (or second stage), the Mask R-CNN approach is recommended. If YOLO yields improvement but perhaps not to the level we want (e.g., maybe it still produces a couple of FPs that are hard to threshold out without losing recall), a Mask R-CNN could further refine accuracy. This backup is essentially our **"precision specialist"** – we would invest the training time to get a model that understands barline shape so well that almost no false stem or noise will fool it. Because it's more complex, I'd only pursue it if needed (or in parallel if time allows). The license and integration for Mask R-CNN are favorable (Detectron2 is solid). The caveat is training data preparation and training time, but those are manageable. We should monitor if our dataset choices are enough – if purely synthetic is not, we incorporate the fine-tuning on real data (Candidate 3) into this.

In summary, my plan would be: **Deploy the YOLO detector on our test page (and others) to quantify FP reduction.** If it's already excellent (e.g., FPs drop from 30 to <5 with no FN introduced), we might stick with that solution (possibly retraining it under an appropriate license). If we find edge cases where YOLO still struggles (maybe a faint barline it missed, or a specific symbol it incorrectly flags as barline), then **train a Mask R-CNN** on the same data (plus augmentations) and compare. Mask R-CNN should handle those edge cases better, at the cost of more implementation work.

Finally, incorporating some **real-data fine-tuning** is advisable once we have either model running, to cement its performance on actual scans. This could be done as a later step in either case.

By following this recommendation, we leverage existing work and data (no new manual annotation), respect licensing for future use, and maintain a modular pipeline (the new model can simply replace the old barline stage). The primary candidate (YOLO) gives us quick wins, and the backup (Mask R-CNN) is there to push towards the upper bound of accuracy if needed.

---

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 41 85 86 87 88 89 90 92 93 Barline Detection & Measure Numbering – Models Comparison.pdf
file://file_00000000ffbc7206aef8f6cb4af95743

30 31 34 36 37 91 DeepScoresV2
https://zenodo.org/records/4012193

32 33 35 38 39 40 42 43 44 45 46 47 48 49 50 51 52 55 58 Optical Music Recognition Datasets | OMR-Datasets
https://apacha.github.io/OMR-Datasets/

53 [2107.07786] DoReMi: First glance at a universal OMR dataset
https://arxiv.org/abs/2107.07786

54 56 57 59 60 61 62 AudioLabs - Tools for Semi-Automatic Bounding Box Annotation of Musical Measures in Sheet Music
https://www.audiolabs-erlangen.de/resources/MIR/2019-ISMIR-LBD-Measures

63 64 65 66 67 68 69 70 71 72 73 74 83 GitHub - dmgonzalez8/OMR
https://github.com/dmgonzalez8/OMR

75 76 77 78 79 80 81 82 84 GitHub - CatUnderTheLeaf/musicScanner: Optical Music Recognition using Deep Learning
https://github.com/CatUnderTheLeaf/musicScanner